

Unit 1. Statistical Learning

1 What Is Statistical Learning?

ISLR §2.1 · FREES §1.1, §1.4–1.6.

1.1 The model and the regression function

Definition 1.1 (Statistical learning model). Let (X, Y) be a random pair, X taking values in $\mathbb{R}^d$ and Y in $\mathbb{R}$. The **statistical learning model** writes

$$Y = f(X) + \varepsilon, \tag{1}$$

where f is a fixed unknown function and ε is independent of X with $\mathbf{E}[\varepsilon] = 0$. **Statistical learning** is the estimation of f from a training sample $\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^n$.

Theorem 1.2 (The regression function is the optimal predictor). Let $\mathbf{E}[Y^2] < \infty$ and set $f^*(x) = \mathbf{E}[Y \mid X = x]$. Then for every measurable $g : \mathbb{R}^d \rightarrow \mathbb{R}$,

$$\mathbf{E}[(Y - f^*(X))^2] \leq \mathbf{E}[(Y - g(X))^2], \tag{2}$$

with equality iff $g(X) = f^*(X)$ almost surely.

Proof. Condition on $X = x$ and insert $\pm f^*(x)$:

$$\begin{aligned}\mathbf{E}[(g(X) - Y)^2 \mid X = x] &= \mathbf{E}[(g(x) - f^*(x) + f^*(x) - Y)^2 \mid X = x] \\ &= (g(x) - f^*(x))^2 + 2(g(x) - f^*(x)) \mathbf{E}[f^*(x) - Y \mid X = x] \\ &\quad + \mathbf{E}[(f^*(X) - Y)^2 \mid X = x].\end{aligned}$$

The middle term vanishes, since $\mathbf{E}[Y \mid X = x] = f^*(x)$ by definition. Hence

$$\mathbf{E}[(g(X) - Y)^2 \mid X = x] = (g(x) - f^*(x))^2 + \mathbf{E}[(f^*(X) - Y)^2 \mid X = x] \quad (3)$$

and the first term is non-negative. Taking expectations over X gives (2), with equality iff $\mathbf{E}[(g(X) - f^*(X))^2] = 0$. $\square$

Remark. The f of (1) is therefore f^* , and $\varepsilon = Y - f^*(X)$ has conditional mean zero by construction.

Definition 1.3 (Reducible and irreducible error). With $\hat{f}$ an estimate of f and $\hat{Y} = \hat{f}(X)$,

$$\mathbb{E}[(Y - \hat{Y})^2 \mid X = x] = \underbrace{(f(x) - \hat{f}(x))^2}_{\text{reducible}} + \underbrace{\text{var}(\varepsilon)}_{\text{irreducible}}. \quad (4)$$

Remark. $\text{var}(\varepsilon) > 0$ whenever ε carries variables that were not measured, or variation that is unmeasurable. It is unknown in practice, since computing it requires the f that is being estimated.

1.2 Prediction and inference

Remark (Two purposes). In **prediction** only the accuracy of $\hat{Y}$ matters and $\hat{f}$ may be a black box. In **inference** the object of interest is f itself: which coordinates of X enter, with what sign and magnitude, and whether a linear summary is adequate.

1.3 Parametric and non-parametric methods

Definition 1.4 (Parametric, non-parametric). A **parametric** method assumes a functional form for f indexed by finitely many parameters, for example $f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_d X_d$, and estimates those parameters. A **non-parametric** method assumes no such form, seeking only an $\hat{f}$ close to the data subject to a smoothness or complexity restriction.

Remark. A parametric method replaces estimation of a function by estimation of $d + 1$ numbers, at the risk that the assumed form excludes the truth. A non-parametric method carries no such risk and needs far more data, nothing being borrowed from an assumed shape. Flexibility and interpretability move in opposite directions along the same axis: least squares is the least flexible and the most interpretable, boosting and neural networks the reverse.

1.4 Classes of problem

Remark (Terminology). Learning is **supervised** when a response Y is observed alongside the predictors and **unsupervised** when it is not; principal components and clustering (Unit 13) are the unsupervised methods on the syllabus. A supervised problem is a **regression** problem when Y is quantitative and a **classification** problem when Y is qualitative. The distinction is in the response, not the predictors: qualitative predictors are handled by binary coding in a regression (Unit 3), and logistic regression is a classification method despite its name (Unit 7).

1.5 Sampling assumptions and study design

Definition 1.5 (Basic sampling assumptions). For a sample $\{y_1, \dots, y_n\}$ drawn from a population:

- (S1) $E[y_i] = \mu$;
- (S2) $\text{var}(y_i) = \sigma^2$;
- (S3) the y_i are independent;
- (S4) the y_i are normally distributed.

Remark. (S4) is not needed for most inference, because central limit theorems give approximate normality of averages; it is needed for exact small-sample statements such as the t distribution. Regression coefficients are weighted averages (Unit 2), so the same central limit results apply to them. Where the response is badly skewed, a power transform, most often the logarithm, brings it closer to symmetry before the normal approximation is invoked.

Remark (Designed and observational data). In a **designed** study the analyst controls the assignment of treatments, and randomisation licenses causal conclusions. In an **observational** study the data are not under the analyst's control; a confounding variable related to both y and x can generate association without causation, and only **statistical** control, meaning inclusion of the confounder as an explanatory variable, is available. Actuarial data are almost always observational, so the output of a regression is an association, adjusted for whatever was measured. Unit 3 returns to this when interpreting coefficients.

2 Assessing Model Accuracy

ISLR §2.2.

2.1 Quality of fit

Definition 1.6 (Training and test error). For a fit $\hat{f}$ obtained from $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$,

$$\text{MSE}_{\text{tr}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2, \quad (5)$$

while the **test** (or generalisation) error is $\mathbf{E}[(Y_0 - \hat{f}(X_0))^2]$ for a pair (X_0, Y_0) drawn from the same population and independent of $\mathcal{D}$.

Remark. $\hat{f}$ was chosen to make (5) small, so MSE_{tr} is a downward-biased estimate of test error; the gap is the **optimism** of the training error and it widens with flexibility. The method with the smallest training error need not have the smallest test error.

2.2 The classification setting

Let $Y \in \{0, 1\}$ and write

$$\eta(x) = \mathbf{P}\{Y = 1 \mid X = x\} = \mathbf{E}[Y \mid X = x]. \quad (6)$$

Definition 1.7 (Classifier, error rate). A **classifier** is a function $g : \mathbb{R}^d \rightarrow \{0, 1\}$; its **error probability** is $L(g) = \mathbf{P}\{g(X) \neq Y\}$. On a sample the error rate is $n^{-1} \sum_i \mathbb{1}_{\{y_i \neq g(x_i)\}}$.

Definition 1.8 (Bayes classifier). The **Bayes classifier** is

$$g^*(x) = \begin{cases} 1 & \eta(x) > 1/2, \\ 0 & \text{otherwise,} \end{cases} \quad (7)$$

and $L^* = \mathbf{P}\{g^*(X) \neq Y\}$ is the **Bayes error**.

Theorem 1.9 (Optimality of the Bayes classifier). For every classifier $g : \mathbb{R}^d \rightarrow \{0, 1\}$,

$$\mathbf{P}\{g^*(X) \neq Y\} \leq \mathbf{P}\{g(X) \neq Y\}.$$

Proof. Conditionally on $X = x$, for any g ,

$$\begin{aligned} \mathbf{P}\{g(X) \neq Y \mid X = x\} &= 1 - \mathbf{P}\{Y = g(X) \mid X = x\} \\ &= 1 - (\mathbf{1}_{\{g(x)=1\}} \eta(x) + \mathbf{1}_{\{g(x)=0\}} (1 - \eta(x))). \end{aligned}$$

Subtracting the same expression for g^* ,

$$\begin{aligned} &\mathbf{P}\{g(X) \neq Y \mid X = x\} - \mathbf{P}\{g^*(X) \neq Y \mid X = x\} \\ &= \eta(x) (\mathbf{1}_{\{g^*(x)=1\}} - \mathbf{1}_{\{g(x)=1\}}) + (1 - \eta(x)) (\mathbf{1}_{\{g^*(x)=0\}} - \mathbf{1}_{\{g(x)=0\}}) \\ &= (2\eta(x) - 1) (\mathbf{1}_{\{g^*(x)=1\}} - \mathbf{1}_{\{g(x)=1\}}) \geq 0, \end{aligned}$$

because $g^*(x) = 1$ exactly when $2\eta(x) - 1 > 0$, so the two factors carry the same sign. Integrating over the distribution of X gives the claim. $\square$

Corollary 1.10. $L^* = \mathbf{E}[\min\{\eta(X), 1 - \eta(X)\}]$.

Proof. The display in the proof gives $\mathbf{P}\{g^*(X) \neq Y \mid X = x\} = 1 - \max\{\eta(x), 1 - \eta(x)\} = \min\{\eta(x), 1 - \eta(x)\}$; integrate. $\square$

Remark. $L^* > 0$ whenever $0 < \eta(x) < 1$ on a set of positive probability, that is, whenever the classes overlap. It is the classification counterpart of the irreducible error of (4), and like it unknown in practice: η is exactly what is being estimated.

2.3 The K -nearest neighbour classifier

Definition 1.11 (KNN). Given K and a point x_0 , let $\mathcal{N}_0$ be the set of indices of the K training points nearest x_0 . The **KNN classifier** estimates

$$\hat{\mathbf{P}}\{Y = j \mid X = x_0\} = \frac{1}{K} \sum_{i \in \mathcal{N}_0} \mathbf{1}_{\{y_i = j\}} \quad (8)$$

and assigns x_0 to the class with the largest estimated probability.

Remark. KNN estimates η by a local average and then applies the Bayes rule to the estimate. $1/K$ measures flexibility: $K = 1$ gives a partition with zero training error and high variance, and large K gives an almost constant, low-variance, high-bias rule.

End of period 1 of 3.

3 The Bias–Variance Decomposition

ISLR §2.2.2.

Throughout, $\mathcal{D}$ is the training sample, $\hat{f} = \hat{f}_{\mathcal{D}}$ the predictor fitted on it, and (X_0, Y_0) a test pair independent of $\mathcal{D}$. Subscripts on $\mathbf{E}$ name the source averaged over.

Definition 1.12 (Pointwise bias and variance). At a fixed x ,

$$\text{bias}(\hat{f}, x) = \mathbf{E}_{\mathcal{D}}[\hat{f}(x)] - f(x), \quad \text{var}(\hat{f}, x) = \text{var}_{\mathcal{D}}(\hat{f}(x)).$$

Theorem 1.13 (Bias–variance decomposition). With $\text{var}(\varepsilon) = \sigma^2$, for a fixed x_0 ,

$$\mathbb{E}[(Y_0 - \hat{f}(x_0))^2] = \text{var}(\hat{f}, x_0) + [\text{bias}(\hat{f}, x_0)]^2 + \sigma^2 \quad (9)$$

where the expectation is over both $\mathcal{D}$ and Y_0 .

Proof. Step 1: separate the noise. Write $Y_0 - \hat{f}(x_0) = (Y_0 - f(x_0)) + (f(x_0) - \hat{f}(x_0))$, square, and take expectations. The first square is σ^2 . The cross term vanishes: $\mathcal{D}$ is independent of Y_0 , so the expectation factors, and $\mathbb{E}[Y_0 - f(x_0)] = 0$. Hence

$$\mathbb{E}[(Y_0 - \hat{f}(x_0))^2] = \sigma^2 + \mathbb{E}_{\mathcal{D}}[(f(x_0) - \hat{f}(x_0))^2].$$

Step 2: split the remaining term. Insert $\pm \mathbb{E}_{\mathcal{D}}[\hat{f}(x_0)]$:

$$f(x_0) - \hat{f}(x_0) = \underbrace{(f(x_0) - \mathbb{E}_{\mathcal{D}} \hat{f}(x_0))}_{= -\text{bias, constant}} + \underbrace{(\mathbb{E}_{\mathcal{D}} \hat{f}(x_0) - \hat{f}(x_0))}_{\text{mean zero}}.$$

Squaring and taking $\mathbb{E}_{\mathcal{D}}$, the cross term vanishes (a constant times a mean-zero variable), leaving $[\text{bias}(\hat{f}, x_0)]^2 + \text{var}(\hat{f}, x_0)$. $\square$

Corollary 1.14. $E[(Y_0 - \hat{f}(x_0))^2] \geq \sigma^2$ for every $\hat{f}$.

Proof. Variance and squared bias are non-negative. □

Remark. Increasing flexibility lowers bias and raises variance, so the sum in (9) is typically U-shaped in flexibility and the best method is neither the most nor the least flexible. Which term dominates depends on the truth: if f is nearly linear, a flexible method buys almost no bias reduction and pays the variance anyway.

4 Resampling

ISLR §5.1–5.3. §5.2 (the bootstrap) is outside the SRM reading list; it is taken here because bagging needs it.

Resampling holds part of the training data out, fits on the remainder, and predicts the held-out part; it is how test error is estimated when no test set is held in reserve.

4.1 The validation-set approach

Definition 1.15 (Validation set). Split the n observations at random into a training half and a validation half; fit on the first and report the mean squared error on the second.

Remark. Two defects. The estimate is highly variable, since it depends on which observations happened to fall in the validation set; and it is fitted on roughly $n/2$ observations, so it **overstates** the test error of the model that would be fitted to all n .

4.2 Leave-one-out cross-validation

Definition 1.16 (LOOCV). Let $\hat{f}^{(-i)}$ be the fit obtained with observation i omitted. Then

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i, \quad \text{MSE}_i = \left(y_i - \hat{f}^{(-i)}(x_i) \right)^2. \quad (10)$$

Remark. LOOCV has almost no bias, each fit using $n - 1$ observations, and it is deterministic: there is no random split, so repeating it gives the same number. Its cost is n fits, except for least

squares, where one fit suffices.

Lemma 1.17 (Sherman–Morrison). Let $\mathbf{A}$ be invertible and $\mathbf{u}, \mathbf{v}$ vectors with $1 - \mathbf{v}'\mathbf{A}^{-1}\mathbf{u} \neq 0$. Then

$$(\mathbf{A} - \mathbf{u}\mathbf{v}')^{-1} = \mathbf{A}^{-1} + \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}'\mathbf{A}^{-1}}{1 - \mathbf{v}'\mathbf{A}^{-1}\mathbf{u}}. \quad (11)$$

Proof. Multiply the right-hand side by $\mathbf{A} - \mathbf{u}\mathbf{v}'$ and write $c = 1 - \mathbf{v}'\mathbf{A}^{-1}\mathbf{u}$:

$$\mathbf{I} - \mathbf{A}^{-1}\mathbf{u}\mathbf{v}' + \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}' - \mathbf{A}^{-1}\mathbf{u}(\mathbf{v}'\mathbf{A}^{-1}\mathbf{u})\mathbf{v}'}{c} = \mathbf{I} - \mathbf{A}^{-1}\mathbf{u}\mathbf{v}' + \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}'(1 - \mathbf{v}'\mathbf{A}^{-1}\mathbf{u})}{c} = \mathbf{I},$$

using that $\mathbf{v}'\mathbf{A}^{-1}\mathbf{u}$ is a scalar. □

Theorem 1.18 (LOOCV for least squares). For the linear model fitted by least squares, with $\hat{y}_i$

the i -th fitted value from the fit on all n observations and $h_{ii} = \mathbf{x}_i'(\mathbf{X}'\mathbf{X})^{-1}\mathbf{x}_i$ the i -th leverage,

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_{ii}} \right)^2. \quad (12)$$

A single fit therefore delivers the n -fold cross-validated error.

Proof. Deleting observation i replaces $\mathbf{X}'\mathbf{X}$ by $\mathbf{X}'\mathbf{X} - \mathbf{x}_i\mathbf{x}_i'$ and $\mathbf{X}'\mathbf{y}$ by $\mathbf{X}'\mathbf{y} - \mathbf{x}_iy_i$. By Lemma 1.17 with $\mathbf{A} = \mathbf{X}'\mathbf{X}$ and $\mathbf{u} = \mathbf{v} = \mathbf{x}_i$, so that $\mathbf{v}'\mathbf{A}^{-1}\mathbf{u} = h_{ii}$,

$$(\mathbf{X}'\mathbf{X} - \mathbf{x}_i\mathbf{x}_i')^{-1} = (\mathbf{X}'\mathbf{X})^{-1} + \frac{(\mathbf{X}'\mathbf{X})^{-1}\mathbf{x}_i\mathbf{x}_i'(\mathbf{X}'\mathbf{X})^{-1}}{1 - h_{ii}}.$$

Write $\hat{\boldsymbol{\beta}}$ for the full-sample estimate and $\hat{\boldsymbol{\beta}}^{(-i)}$ for the deleted one. Multiplying the display by $\mathbf{X}'\mathbf{y} - \mathbf{x}_iy_i$ and collecting terms,

$$\hat{\boldsymbol{\beta}}^{(-i)} = \hat{\boldsymbol{\beta}} - (\mathbf{X}'\mathbf{X})^{-1}\mathbf{x}_i \left[y_i - \frac{\hat{y}_i - h_{ii}y_i}{1 - h_{ii}} \right] = \hat{\boldsymbol{\beta}} - (\mathbf{X}'\mathbf{X})^{-1}\mathbf{x}_i \frac{y_i - \hat{y}_i}{1 - h_{ii}},$$

the bracket simplifying because $y_i(1 - h_{ii}) - \hat{y}_i + h_{ii}y_i = y_i - \hat{y}_i$. Premultiplying by $\mathbf{x}_i'$,

$$\hat{f}^{(-i)}(x_i) = \mathbf{x}_i' \hat{\boldsymbol{\beta}}^{(-i)} = \hat{y}_i - h_{ii} \frac{y_i - \hat{y}_i}{1 - h_{ii}},$$

so that

$$y_i - \hat{f}^{(-i)}(x_i) = (y_i - \hat{y}_i) \left(1 + \frac{h_{ii}}{1 - h_{ii}} \right) = \frac{y_i - \hat{y}_i}{1 - h_{ii}}.$$

Squaring and averaging gives (12). □

Remark. The formula is the ordinary training residual with the i -th residual inflated by $1/(1 - h_{ii})$. High-leverage points pull the fitted surface toward themselves, so their training residuals understate their true prediction error by exactly this factor, the same h_{ii} that Unit 2 shows satisfies $\text{var}(e_i) = \sigma^2(1 - h_{ii})$. Since $1 - h_{ii} \leq 1$, $\text{CV}_{(n)} \geq \text{MSE}_{\text{tr}}$ term by term.

4.3 k -fold cross-validation

Definition 1.19 (k -fold CV). Partition the observations at random into k folds of roughly equal size. For $j = 1, \dots, k$, fit on all folds but the j -th and compute MSE_j on the j -th. Then

$$\text{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^k \text{MSE}_j. \quad (13)$$

Remark (Choosing k). k -fold costs k fits rather than n . It is also often **more accurate** than LOOCV, for a bias–variance reason of its own. Each of the n LOOCV fits uses almost the same $n - 1$ observations, so the n quantities averaged are highly positively correlated, and the mean of highly correlated quantities has high variance; the k fits in k -fold overlap less. Against that, each k -fold fit uses only $(k - 1)n/k$ observations, so $\text{CV}_{(k)}$ is biased upward more than $\text{CV}_{(n)}$. Taking $k = 5$ or $k = 10$ is the usual compromise. Unlike LOOCV, k -fold depends on the random split, so it varies from run to run.

Remark (Classification). For a qualitative response, replace MSE_i in (10) by the misclassification indicator $\mathbb{1}_{\{y_i \neq \hat{y}_i^{(-i)}\}}$; everything else is unchanged. The shortcut of Theorem 1.18 is specific to least squares and does not carry over.

End of period 2 of 3.

4.4 The bootstrap

Definition 1.20 (Bootstrap sample). Given data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, a **bootstrap sample** $\mathcal{D}^*$ is obtained by drawing n indices independently and uniformly with replacement from $\{1, \dots, n\}$. An index may be drawn several times or not at all.

Definition 1.21 (Bootstrap standard error). Let $\hat{\theta}$ be a statistic computed from $\mathcal{D}$, and $\hat{\theta}^{*b}$ its

value on the b -th of B bootstrap samples. The **bootstrap standard error** is

$$\text{SE}_B(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B \left(\hat{\theta}^{*b} - \frac{1}{B} \sum_{c=1}^B \hat{\theta}^{*c} \right)^2}. \quad (14)$$

Proposition 1.22 (Out-of-bag fraction). The probability that a given observation does not appear in a bootstrap sample is $(1 - 1/n)^n$, which increases to $e^{-1} \approx 0.368$ as $n \rightarrow \infty$.

Proof. Each of the n independent draws misses observation i with probability $1 - 1/n$, so all n draws miss it with probability $(1 - 1/n)^n$. Writing $(1 - 1/n)^n = \exp\{n \ln(1 - 1/n)\}$ and $\ln(1 - u) = -u - u^2/2 - \dots$ gives $n \ln(1 - 1/n) \rightarrow -1$. $\square$

Remark. Those **out-of-bag** observations, roughly a third of the sample, are what make bagging's free error estimate possible (Unit 12).

Remark (Bootstrap against cross-validation). Cross-validation estimates **test error**; the bootstrap estimates the **sampling variability of a statistic**. The bootstrap is the tool when a standard error is wanted and no formula exists, as for a median, a ratio of coefficients or a tuning parameter selected by CV, and when a formula exists but rests on assumptions in doubt: the regression standard error $s\sqrt{[(\mathbf{X}'\mathbf{X})^{-1}]_{jj}}$ assumes a correctly specified model with constant variance, resampling the observed pairs assumes neither, and a gap between the two is itself a diagnostic.

5 Computation: Formulas Against Package Output

ISLR §2.3, §5.3 (excluding §5.3.4).

Several quantities here, namely f , σ^2 , η and L^* , are unknowable from data, so the checks that involve them are run on simulations where the truth is set by us.

```
1 f      <- function(x) sin(2 * x) + 0.5 * x      # the true regression function
2 sigma <- 0.5                                     # sd of the noise
3 eta    <- function(x) plogis(2 * x)             # true  $P(Y = 1 \mid X = x)$ 
```

5.1 The regression function and the irreducible error (§1.1)

Theorem 1.2 says no predictor beats f , and (4) says its error is exactly $\text{var}(\varepsilon)$. Both are checkable when f is known.

```

1 set.seed(1)
2 N <- 200000
3 x <- runif(N, -3, 3)
4 y <- f(x) + rnorm(N, sd = sigma)
5 c(mse_of_true_f = mean((y - f(x))^2),           # attains the irreducible error
6   sigma_squared = sigma^2,
7   mse_of_linear = mean((y - predict(lm(y ~ x)))^2), # any other g does worse
8   mse_of_mean   = mean((y - mean(y))^2))

```

```

## mse_of_true_f sigma_squared mse_of_linear mse_of_mean
##           0.2497           0.2500           0.6867           1.0181

```

A power transform (FREES §1.3) is judged by how much closer to symmetry it brings the response; on a strongly right-skewed variable the logarithm removes most of the skew.

```

1 skew <- function(v) mean((v - mean(v))^3) / sd(v)^3
2 c(raw_medv = skew(Boston$medv), log_medv = skew(log(Boston$medv)),

```

```
3 raw_crim = skew(Boston$crim), log_crim = skew(log(Boston$crim)))
```

```
## raw_medv log_medv raw_crim log_crim
## 1.1015 -0.3284 5.1922 0.4035
```

5.2 The Bayes classifier and the Bayes error (§2.2)

With $X \sim U(-3, 3)$ and $\eta(x) = (1 + e^{-2x})^{-1}$, Corollary 1.10 can be evaluated in closed form: by symmetry

$$L^* = \frac{2}{6} \int_0^3 \frac{e^{-2x}}{1 + e^{-2x}} dx = \frac{1}{6} \left[\ln 2 - \ln(1 + e^{-6}) \right].$$

Four routes to the same number: the closed form, numerical integration, the plug-in formula on a sample, and the realised error rate of g^* .

```
1 set.seed(2)
2 xb <- runif(N, -3, 3)
3 yb <- rbinom(N, 1, eta(xb))
4 g_star <- as.integer(eta(xb) > 0.5)           # the Bayes rule
5 L_star <- (log(2) - log(1 + exp(-6))) / 6     # closed form
6 c(closed_form = L_star,
7   integrate = integrate(function(x) pmin(eta(x), 1 - eta(x))/6, -3, 3)$value,
```

```

8  plug_in      = mean(pmin(eta(xb), 1 - eta(xb))),      #  $E[\min(\eta, 1-\eta)]$ 
9  realised     = mean(g_star != yb))                  # error rate of  $g^*$ 

```

```

## closed_form  integrate    plug_in    realised
##      0.1151      0.1151      0.1147      0.1146

```

Theorem 1.9 says no rule does better. Three competitors, none of which beats g^* :

```

1  c(bayes      = mean(g_star != yb),
2    threshold_1 = mean(as.integer(xb > 1) != yb),
3    threshold_m1 = mean(as.integer(xb > -1) != yb),
4    always_one  = mean(1 != yb))

```

```

##          bayes  threshold_1 threshold_m1  always_one
##      0.1146      0.1879      0.1859      0.4980

```

5.3 KNN and the role of K (§2.3)

```

1  set.seed(3)
2  ntr <- 200; nte <- 50000
3  tr <- data.frame(x = runif(ntr, -3, 3)); tr$y <- rbinom(ntr, 1, eta(tr$x))
4  te <- data.frame(x = runif(nte, -3, 3)); te$y <- rbinom(nte, 1, eta(te$x))
5  Ks <- c(1, 3, 5, 15, 45, 105)

```

```

6 err <- t(sapply(Ks, function(K) {
7   ktr <- class::knn(tr["x"], tr["x"], tr$y, k = K)
8   kte <- class::knn(tr["x"], te["x"], tr$y, k = K)
9   c(train = mean(ktr != tr$y), test = mean(kte != te$y))
10 })))
11 mc_se <- sqrt(L_star * (1 - L_star) / nte)
12 cbind(K = Ks, round(err, 4), bayes = round(L_star, 4),
13       mc_se = round(mc_se, 4), z = round((err[, "test"] - L_star)/mc_se, 2))

```

```

##      K train  test  bayes  mc_se    z
## [1,]  1 0.000 0.1715 0.1151 0.0014 39.52
## [2,]  3 0.090 0.1293 0.1151 0.0014  9.93
## [3,]  5 0.125 0.1295 0.1151 0.0014 10.07
## [4,] 15 0.130 0.1134 0.1151 0.0014 -1.23
## [5,] 45 0.130 0.1133 0.1151 0.0014 -1.28
## [6,] 105 0.135 0.1145 0.1151 0.0014 -0.41

```

At $K = 1$ the training error is zero and the test error is the worst in the table; the test error is smallest at an intermediate K .

Remark. Theorem 1.9 bounds the **population** error probability, not the error rate realised on a finite test sample. The latter averages n_{te} indicators and has standard error $\sqrt{L^*(1 - L^*)/n_{\text{te}}}$, printed as `mc_se`; the `z` column reports how many of those standard errors separate each row from L^* , and the rows below L^* are within sampling error of it. An empirical rate that appears to beat a theoretical bound should be compared with this quantity before anything else.

5.4 Training error versus test error (§2.1)

```

1 set.seed(4)
2 dtr <- data.frame(x = runif(80, -3, 3)); dtr$y <- f(dtr$x) + rnorm(80, sd = sigma)
3 dte <- data.frame(x = runif(20000, -3, 3)); dte$y <- f(dte$x) + rnorm(20000, sd = sigma)
4 degs <- c(1, 2, 4, 6, 8, 12, 16)
5 tt <- t(sapply(degs, function(dg) {
6   m <- lm(y ~ poly(x, dg), data = dtr)
7   c(train = mean(residuals(m)^2),
8     test  = mean((dte$y - predict(m, newdata = dte))^2))
9 })))
10 cbind(degree = degs, round(tt, 4), irreducible = sigma^2)

```

```
##      degree  train   test irreducible
```

## [1,]	1	0.6158	0.6836	0.25
## [2,]	2	0.6031	0.6976	0.25
## [3,]	4	0.4825	0.5792	0.25
## [4,]	6	0.2393	0.2876	0.25
## [5,]	8	0.2066	0.2593	0.25
## [6,]	12	0.2025	0.2631	0.25
## [7,]	16	0.1837	0.3245	0.25

The training error falls monotonically in flexibility. The test error reaches its minimum at degree 8 and rises after it, and stays above σ^2 throughout, as Corollary 1.14 requires.

5.5 The bias–variance decomposition (§3)

Equation (9) is an identity between a quantity that can be simulated directly and three that must be assembled separately. Both sides are computed below at the fixed point $x_0 = 1$, for a range of polynomial degrees.

```
1 set.seed(5)
2 x0 <- 1.0; B <- 1500; nsim <- 60
3 bv <- t(apply(degs[1:6], function(dg) {
4   fh <- y0 <- numeric(B)
5   for (b in seq_len(B)) {
```

```

6      d <- data.frame(x = runif(nsim, -3, 3))
7      d$y <- f(d$x) + rnorm(nsim, sd = sigma)
8      fh[b] <- predict(lm(y ~ poly(x, dg), data = d), newdata = data.frame(x = x0))
9      y0[b] <- f(x0) + rnorm(1, sd = sigma)
10    }
11    b2 <- (mean(fh) - f(x0))^2; v <- var(fh)
12    c(bias2 = b2, variance = v, irreducible = sigma^2,
13      sum = b2 + v + sigma^2, direct = mean((y0 - fh)^2))
14  )))
15 cbind(degree = degs[1:6], round(bv, 4))

```

##	degree	bias2	variance	irreducible	sum	direct
## [1,]	1	1.1709	0.0142	0.25	1.4351	1.4233
## [2,]	2	1.1589	0.0261	0.25	1.4350	1.4637
## [3,]	4	0.4212	0.0333	0.25	0.7044	0.6997
## [4,]	6	0.0081	0.0259	0.25	0.2839	0.2940
## [5,]	8	0.0000	0.0295	0.25	0.2795	0.2854
## [6,]	12	0.0000	0.0436	0.25	0.2936	0.3048

sum and **direct** agree to Monte-Carlo error at every degree, which is Theorem 1.13; reading down the columns, the squared bias falls to zero by degree 8 while the variance grows from 0.014 to 0.044,

which is the trade-off.

5.6 LOOCV: the shortcut against the definition (§4.2)

Theorem 1.18 claims that one fit reproduces n fits. Both are computed, on the same model Unit 2 uses, and compared with `boot::cv.glm`.

```

1 fit <- lm(medv ~ lstat + age + rm, data = Boston)
2 n     <- nobs(fit); h <- hatvalues(fit); e <- residuals(fit)
3
4 cv_shortcut <- mean((e / (1 - h))^2)                # one fit
5 loo_pred    <- sapply(seq_len(n), function(i)      # n fits
6   predict(lm(medv ~ lstat + age + rm, data = Boston[-i, ]), Boston[i, ]))
7 cv_brute    <- mean((Boston$medv - loo_pred)^2)
8
9 suppressMessages(library(boot))
10 cv_pkg <- cv.glm(Boston, glm(medv ~ lstat + age + rm, data = Boston))$delta[1]
11 c(shortcut = cv_shortcut, brute_force = cv_brute, cv.glm = cv_pkg)

```

```
##      shortcut brute_force      cv.glm
##      31.5      31.5      31.5
```

The per-observation identity behind the theorem, not merely its average:

```

1 c(max_abs_diff = max(abs((Boston$medv - loo_pred) - e / (1 - h))),
2   training_MSE = mean(e^2), LOOCV = cv_shortcut,
3   ratio        = cv_shortcut / mean(e^2))

```

```

## max_abs_diff training_MSE      LOOCV      ratio
##      1.918e-13      3.047e+01      3.150e+01      1.034e+00

```

$CV_{(n)} > MSE_{tr}$, as the remark after Theorem 1.18 predicts from $1 - h_{ii} \leq 1$.

5.7 k -fold cross-validation (§4.3)

```

1 cv_kfold <- function(K) {
2   folds <- sample(rep(seq_len(K), length.out = n))
3   mean(vapply(seq_len(K), function(j) {
4     m <- lm(medv ~ lstat + age + rm, data = Boston[folds != j, ])
5     mean((Boston$medv[folds == j] - predict(m, Boston[folds == j, ]))^2)
6   }, numeric(1)))
7 }
8 set.seed(6)
9 r5  <- replicate(30, cv_kfold(5))
10 r10 <- replicate(30, cv_kfold(10))

```

```

11 rbind(fivefold = c(mean = mean(r5), sd = sd(r5)),
12      tenfold  = c(mean = mean(r10), sd = sd(r10)),
13      loocv     = c(mean = cv_shortcut, sd = 0))

```

```

##           mean      sd
## fivefold 31.65 0.3851
## tenfold  31.54 0.3257
## loocv    31.50 0.0000

```

5.8 The bootstrap (§4.4)

```

1 nB <- nobs(fit)
2 set.seed(7)
3 oob <- mean(replicate(2000, {
4   idx <- sample(nB, nB, replace = TRUE); mean(!(seq_len(nB) %in% idx))
5 })))
6 c(empirical = oob, formula = (1 - 1/nB)^nB, limit = exp(-1))

```

```

## empirical  formula      limit
##    0.3677    0.3675    0.3679

```

The bootstrap standard error of each coefficient, against the formula $s\sqrt{[(\mathbf{X}'\mathbf{X})^{-1}]_{jj}}$:

```
1 boot_coef <- function(d, i) coef(lm(medv ~ lstat + age + rm, data = d[i, ]))
2 set.seed(7)
3 bo <- boot(Boston, boot_coef, R = 1000)
4 round(rbind(formula      = coef(summary(fit))[, "Std. Error"],
5             bootstrap    = apply(bo$t, 2, sd),
6             ratio        = apply(bo$t, 2, sd) / coef(summary(fit))[, "Std. Error"]), 4)
```

```
##              (Intercept)  lstat    age    rm
## formula              3.182 0.0544 0.0112 0.4543
## bootstrap              5.346 0.0972 0.0167 0.7949
## ratio                  1.680 1.7886 1.4914 1.7498
```

The bootstrap standard errors are the larger, by factors of 1.5 to 1.8. The formula assumes a correctly specified model with constant variance; resampling pairs assumes neither. The gap is evidence against those assumptions, not against the bootstrap, and Unit 4 measures the same thing a second way, through heteroscedasticity-consistent standard errors.

6 Pitfalls

6.1 Reporting the training error as performance

The training error can be driven to zero by flexibility alone and says nothing about test error. A 1-nearest-neighbour classifier is the extreme case.

```
1 k1_train <- class::knn(tr["x"], tr["x"], tr$y, k = 1)
2 k1_test  <- class::knn(tr["x"], te["x"], tr$y, k = 1)
3 c(train_error = mean(k1_train != tr$y),          # exactly zero
4   test_error  = mean(k1_test  != te$y),
5   bayes_error = mean(pmin(eta(te$x), 1 - eta(te$x))))
```

```
## train_error test_error bayes_error
##          0.0000          0.1715          0.1152
```

Diagnostic. A training error far below any plausible irreducible error. **Fix.** Report cross-validated or held-out error.

6.2 Selecting predictors before cross-validating

If predictors are screened on the full data and cross-validation is applied afterwards, the held-out fold has already influenced the selection, and the resulting estimate is badly optimistic. Below, the response is independent of all 500 predictors, so the honest error rate is that of guessing.

```
1 set.seed(7)
2 nL <- 100; pL <- 500
3 X1 <- matrix(rnorm(nL * pL), nL, pL)
4 y1 <- rnorm(nL)                                # independent of every column
5
6 best <- order(abs(cor(X1, y1)), decreasing = TRUE)[1:5]  # screened on ALL the data
7
8 cv_fixed <- function(cols, K = 10) {              # columns chosen once, outside
9   folds <- sample(rep(seq_len(K), length.out = nL))
10  mean(vapply(seq_len(K), function(j) {
11    d <- data.frame(y = y1[folds != j], X1[folds != j, cols, drop = FALSE])
12    nd <- data.frame(X1[folds == j, cols, drop = FALSE])
13    mean((y1[folds == j] - predict(lm(y ~ ., data = d), newdata = nd))^2)
14  }, numeric(1)))
15 }
16 cv_inside <- function(K = 10) {                  # screening repeated per fold
17   folds <- sample(rep(seq_len(K), length.out = nL))
18   mean(vapply(seq_len(K), function(j) {
19     trn <- folds != j
20     sel <- order(abs(cor(X1[trn, ], y1[trn])), decreasing = TRUE)[1:5]
```

```

21     d    <- data.frame(y = yl[trn], Xl[trn, sel, drop = FALSE])
22     nd   <- data.frame(Xl[!trn, sel, drop = FALSE])
23     mean((yl[!trn] - predict(lm(y ~ ., data = d), newdata = nd))^2)
24   }, numeric(1)))
25 }
26 set.seed(8)
27 c(screened_on_all_data      = cv_fixed(best),
28   screened_inside_the_fold = cv_inside(),
29   variance_of_y            = var(yl))

```

```

##      screened_on_all_data screened_inside_the_fold
##                0.7424                1.3203
##      variance_of_y
##                0.9775

```

Screening outside the fold reports an error **below** the variance of the response, on data with no signal at all. Screening inside reports one **above** it, correctly, since fitting five noise predictors costs something.

Diagnostic. A cross-validated error below $\text{var}(y)$ when no predictor plausibly carries signal. **Fix.** As above: the object being cross-validated must be the whole procedure, screening included. Unit 5

does this for variable selection and Unit 6 for the choice of λ .

6.3 Cross-validating the wrong unit

Cross-validation assumes the held-out fold is independent of the training folds. When observations are grouped, with several claims from one policyholder or several years from one company, a random split puts members of the same group on both sides and the estimate is optimistic. The same applies to time series, where a random split lets the model train on the future.

Diagnostic. Ask what unit is independent; if it is not the row, a random split is wrong. **Fix.** Split by group, or, for time series, use only past observations to predict future ones (Unit 9).

7 Exercises

Exercise 1.1. Let $\mathbf{E}[Y^2] < \infty$. (a) Show that $f^*(x) = \mathbf{E}[Y \mid X = x]$ is the nearest point to Y , in the L^2 norm, of the space of square-integrable functions of X , and name the result of the prerequisites this is. (b) Assuming only $\mathbf{E} |Y| < \infty$, prove that $\mathbf{E} |Y - g(X)|$ is minimised by any conditional median of Y given $X = x$, without differentiating.

Solution. (a) Theorem 1.2 states $\mathbf{E}[(Y - f^*(X))^2] \leq \mathbf{E}[(Y - g(X))^2]$ for every square-integrable g , so $f^*(X)$ is nearest to Y in $\|Z\|^2 = \mathbf{E}[Z^2]$. That set is a closed subspace, so the nearest point is the orthogonal projection, characterised by $\mathbf{E}[(Y - f^*(X))g(X)] = 0$ for all such g . This is the prerequisites' theorem on conditional expectation as an L^2 projection; Theorem 1.2 is that theorem for squared error.

(b) Fix x and let m be a median of the conditional law, so $\mathbf{P}\{Y \leq m \mid X = x\} \geq 1/2$ and $\mathbf{P}\{Y \geq m \mid X = x\} \geq 1/2$. For $a > m$,

$$|y - a| - |y - m| = \begin{cases} a - m, & y \leq m, \\ a + m - 2y, & m < y < a, \\ m - a, & y \geq a, \end{cases}$$

and $a + m - 2y \geq -(a - m)$ in the middle case because $y \leq a$. So in every case

$$|y - a| - |y - m| \geq (a - m)(2 \mathbf{1}_{\{y \leq m\}} - 1),$$

and taking conditional expectations,

$$\mathbf{E}[|Y - a| - |Y - m| \mid X = x] \geq (a - m)(2\mathbf{P}\{Y \leq m \mid X = x\} - 1) \geq 0.$$

The case $a < m$ is the mirror image. Integrating over X gives the claim; the expectations are finite because $|Y - a| \leq |Y| + |a|$.

Squared error is estimated by a conditional mean, absolute error by a conditional median: the optimal predictor is a property of the loss, not of the data. ■

Exercise 1.2. Under Definition 1.1 with $\text{var}(\varepsilon) = \sigma^2$, use the prerequisites' conditional variance decomposition to split $\text{var}(Y)$, and deduce the largest R^2 any predictor can achieve. Evaluate it for the simulation of §5, where $f(x) = \sin 2x + x/2$, $X \sim \text{U}(-3, 3)$ and $\sigma = 0.5$.

Solution. $\text{var}(Y) = \mathbf{E}[\text{var}(Y \mid X)] + \text{var}(\mathbf{E}[Y \mid X]) = \sigma^2 + \text{var}(f(X))$. Since $R^2 = 1 - \mathbf{E}[(Y - g(X))^2] / \text{var}(Y)$ and the numerator is at least σ^2 by Theorem 1.2,

$$R^2 \leq 1 - \frac{\sigma^2}{\text{var}(Y)} = \frac{\text{var}(f(X))}{\text{var}(f(X)) + \sigma^2}.$$

f is odd and the law of X symmetric, so $\mathbb{E}[f(X)] = 0$ and $\text{var}(f(X)) = \frac{1}{6} \int_{-3}^3 (\sin 2x + x/2)^2 dx$. The three integrals are $\frac{1}{3} \int_0^3 \sin^2 2x dx = \frac{1}{2} - \frac{1}{24} \sin 12$, $\frac{1}{3} \int_0^3 x \sin 2x dx = \frac{1}{12} \sin 6 - \frac{1}{2} \cos 6$ and $\frac{1}{24} \int_{-3}^3 x^2 dx = \frac{3}{4}$, so

$$\text{var}(f(X)) = \frac{5}{4} - \frac{1}{24} \sin 12 + \frac{1}{12} \sin 6 - \frac{1}{2} \cos 6.$$

```
1 vf <- 5/4 - sin(12)/24 + sin(6)/12 - cos(6)/2
2 c(var_f = vf, numeric = integrate(function(x) f(x)^2/6, -3, 3)$value,
3   var_Y = vf + sigma^2, R2_max = vf/(vf + sigma^2))
```

```
##   var_f numeric   var_Y  R2_max
##  0.7690  0.7690  1.0190  0.7547
```

A reported R^2 of 0.85 on data generated this way is impossible, and points to a training-set figure or a leaking split. 

Exercise 1.3. Explain why the irreducible error is strictly positive in almost every application, and why it is unknown. If a colleague reports a test MSE of 0.10 on data you know to have $\text{var}(\varepsilon) = 0.25$, what do you conclude?

Solution. ε collects everything affecting Y that is not a function of the measured X : variables not recorded, and genuinely unpredictable variation. It is unknown because estimating it requires f , which is the problem being solved.

By Corollary 1.14 no method attains an **expected** test MSE below 0.25. A reported 0.10 is therefore not an honest test error: most likely a training error, a leaking split, or a split that ignores dependence in the data (§6). The one innocent explanation is a very small test set: a realised average of squared errors falls below its own expectation about half the time, so ask how many test points the 0.10 came from before concluding anything. ■

Exercise 1.4. Corollary 1.14 bounds an expectation. Suppose $\hat{f} = f$ exactly and the analyst reports $m^{-1} \sum_{i=1}^m (Y_i - f(X_i))^2$ on m independent test points with $\varepsilon \sim N(0, \sigma^2)$. Give the exact distribution of the reported figure, the probability that it falls below σ^2 at $m = 20$ and $m = 20,000$, and its standard deviation. Then judge a reported 0.10 against $\sigma^2 = 0.25$.

Solution. $Y_i - f(X_i) = \varepsilon_i$ are independent $N(0, \sigma^2)$, so $\sum_i \varepsilon_i^2 / \sigma^2 \sim \chi_m^2$ and the reported figure is $(\sigma^2/m) \chi_m^2$, with mean σ^2 and standard deviation $\sigma^2 \sqrt{2/m}$. Hence $\mathbf{P}\{\text{reported} < \sigma^2\} =$

$P\{\chi_m^2 < m\}$, which exceeds $1/2$ for every m because the χ_m^2 density is right-skewed and its median lies below its mean.

```
1 m <- c(20, 20000)
2 rbind(P_below_sigma2 = pchisq(m, m),
3       sd_of_report    = 0.25 * sqrt(2/m),
4       z_of_0.10        = (0.25 - 0.10) / (0.25 * sqrt(2/m)),
5       P_at_or_below    = pchisq(0.10/0.25 * m, m))
```

```
##           [,1]      [,2]
## P_below_sigma2 0.542070 0.5013
## sd_of_report   0.079057 0.0025
## z_of_0.10      1.897367 60.0000
## P_at_or_below  0.008132 0.0000
```

A realised average sits below the floor about half the time however large m is; what shrinks is the size of the shortfall. At $m = 20$ the report is 1.9 standard deviations low, an event of probability 0.008; at $m = 20,000$ it is 60 standard deviations low. Only the first is conceivably chance, and the way to tell is to ask how large the test set was. ■

Exercise 1.5. For a binary response, $\eta(x)$ equals 0.8 on a quarter of the population, 0.3 on a half and 0.55 on the remaining quarter. Give the Bayes classifier and compute the Bayes error, the error of the rule that always predicts 1, and the error of the rule that predicts 1 only where $\eta = 0.8$.

Solution. By (7), g^* predicts 1 where $\eta > 1/2$: on the $\eta = 0.8$ quarter and the $\eta = 0.55$ quarter. By Corollary 1.10,

$$L^* = \frac{1}{4}(0.2) + \frac{1}{2}(0.3) + \frac{1}{4}(0.45) = 0.3125.$$

Always predicting 1: $\frac{1}{4}(0.2) + \frac{1}{2}(0.7) + \frac{1}{4}(0.45) = 0.5125$. Predicting 1 only where $\eta = 0.8$: $\frac{1}{4}(0.2) + \frac{1}{2}(0.3) + \frac{1}{4}(0.55) = 0.3375$. Both exceed L^* , as Theorem 1.9 requires.

The third rule differs from g^* only on the quarter with $\eta = 0.55$, and its penalty there is $\frac{1}{4}(0.55 - 0.45) = 0.025$. The cost of misclassifying a region is proportional to $|2\eta - 1|$, the factor in the proof of Theorem 1.9. 

Exercise 1.6. A false negative costs c_{10} and a false positive c_{01} , both positive. Find the rule minimising expected cost and the minimum cost. Evaluate for $c_{10} = 5$, $c_{01} = 1$ on a population

where η is 0.8 on half and 0.3 on the other half, and compare with the cost incurred by the Bayes classifier.

Solution. $R(g) = c_{10} \mathbf{P}\{g(X) = 0, Y = 1\} + c_{01} \mathbf{P}\{g(X) = 1, Y = 0\}$. Conditionally on $X = x$ the two available costs are $c_{10}\eta(x)$ for $g(x) = 0$ and $c_{01}(1 - \eta(x))$ for $g(x) = 1$, and the choice can be made separately at each x . So the minimiser predicts 1 exactly when $c_{01}(1 - \eta(x)) < c_{10}\eta(x)$, that is

$$\eta(x) > \frac{c_{01}}{c_{01} + c_{10}}, \quad \min_g R(g) = \mathbf{E}[\min\{c_{10}\eta(X), c_{01}(1 - \eta(X))\}].$$

At $c_{10} = c_{01}$ the threshold is $1/2$ and both reduce to Definition 1.8 and Corollary 1.10.

With $c_{10} = 5$, $c_{01} = 1$ the threshold is $1/6$. Both 0.8 and 0.3 exceed it, so the cost-optimal rule predicts 1 everywhere at cost $\frac{1}{2}(1)(0.2) + \frac{1}{2}(1)(0.7) = 0.45$. The Bayes classifier predicts 1 on the 0.8 half and 0 on the 0.3 half, at cost $\frac{1}{2}(1)(0.2) + \frac{1}{2}(5)(0.3) = 0.85$. Its error **rate** is 0.25 against the cost rule's 0.45: the two minimise different criteria. ■

Exercise 1.7. Extend Theorem 1.9 to $Y \in \{1, \dots, c\}$: with $\eta_j(x) = \mathbf{P}\{Y = j \mid X = x\}$, show $g^*(x) = \arg \max_j \eta_j(x)$ minimises $\mathbf{P}\{g(X) \neq Y\}$ and give L^* . Recover Corollary 1.10 at $c = 2$,

compute L^* for $\eta = (0.5, 0.3, 0.2)$ on half the population and $(0.2, 0.2, 0.6)$ on the other half, and give the sharp upper bound on L^* for general c .

Solution. Conditionally on $X = x$, $\mathbf{P}\{g(X) \neq Y \mid X = x\} = 1 - \eta_{g(x)}(x)$, minimised over the c available values by any index attaining $\max_j \eta_j(x)$. The choice is separate at each x , so

$$L^* = \mathbf{E}\left[1 - \max_j \eta_j(X)\right].$$

At $c = 2$ with $\eta_2 = \eta$, $1 - \max\{\eta, 1 - \eta\} = \min\{\eta, 1 - \eta\}$, which is Corollary 1.10.

For the example, $\max_j \eta_j$ is 0.5 then 0.6, so $L^* = 1 - \frac{1}{2}(0.5) - \frac{1}{2}(0.6) = 0.45$.

Since $\max_j \eta_j(x) \geq 1/c$ always, $L^* \leq 1 - 1/c$, attained when every $\eta_j \equiv 1/c$. The familiar bound $L^* \leq 1/2$ is the case $c = 2$ and does not survive: $\eta = (0.4, 0.35, 0.25)$ everywhere gives $L^* = 0.6$.



Exercise 1.8 (ISLR §2.4, exercise 7). The training set below has three predictors and a qualitative response.

Obs.	X_1	X_2	X_3	Y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	-1	0	1	Green
6	1	1	1	Red

- (a) Compute the Euclidean distance from each observation to the test point $X_1 = X_2 = X_3 = 0$.
 (b) Predict with $K = 1$ and with $K = 3$. (c) If the Bayes boundary is highly non-linear, should the best K be large or small? (d) Describe the training and test error as K runs from 1 to n , and say what $K = n$ produces.

Solution.

```

1 Xk <- rbind(c(0,3,0), c(2,0,0), c(0,1,3),
2             c(0,1,2), c(-1,0,1), c(1,1,1))
3 Yk <- c("Red", "Red", "Red", "Green", "Green", "Red")

```

```

4 dk <- sqrt(rowSums(Xk^2))                                # distance to the origin
5 cbind(obs = 1:6, dist = round(dk, 4), Y = Yk)[order(dk), ]

```

```

##      obs dist      Y
## [1,] "5" "1.4142" "Green"
## [2,] "6" "1.7321" "Red"
## [3,] "2" "2"      "Red"
## [4,] "4" "2.2361" "Green"
## [5,] "1" "3"      "Red"
## [6,] "3" "3.1623" "Red"

```

- (a) The distances are 3, 2, $\sqrt{10} = 3.162$, $\sqrt{5} = 2.236$, $\sqrt{2} = 1.414$ and $\sqrt{3} = 1.732$.
- (b) $K = 1$: the nearest is observation 5, so predict **Green**. $K = 3$: observations 5 (Green), 6 (Red) and 2 (Red), so $\hat{\mathbf{P}}\{\text{Red}\} = 2/3$ and the prediction is **Red**. One extra neighbour reverses the answer, which is what makes K the tuning parameter.
- (c) Small. By Definition 1.11 the rule averages η over the K nearest points, so it can only follow a boundary that is roughly constant over that neighbourhood. A highly non-linear boundary needs a small neighbourhood, hence small K , at the cost of the variance the remark after Definition 1.11 describes.

(d) At $K = 1$ every training point is its own neighbour, so the training error is zero and the variance is maximal. As K grows the rule smooths: training error rises, test error falls to a minimum at some intermediate K and then rises. At $K = n$ every point has the same neighbourhood, so the rule is constant at the majority class, with maximal bias and zero variance, and error probability $\min\{\mathbf{P}\{Y = 1\}, \mathbf{P}\{Y = 0\}\}$. At no K can the population error fall below L^* (Theorem 1.9). ■

Exercise 1.9. Let the training pairs be independent with labels drawn from η , and let x' be the nearest training point to a test point x . (a) Show the 1-nearest-neighbour rule errs at x with conditional probability $\eta(x)(1 - \eta(x')) + (1 - \eta(x))\eta(x')$. (b) With η continuous and $x' \rightarrow x$, deduce the limiting risk $R_1 = \mathbf{E}[2\eta(X)(1 - \eta(X))]$. (c) Prove $L^* \leq R_1 \leq 2L^*(1 - L^*) \leq 2L^*$, the middle step by Jensen. (d) Evaluate the chain for the simulation of §5.3 and compare with its $K = 1$ row.

Solution. (a) Given x and x' , the test label Y and the neighbour's label Y' are independent Bernoulli with means $\eta(x)$ and $\eta(x')$. The rule predicts Y' , so it errs exactly when $Y \neq Y'$. (b) Continuity gives $\eta(x') \rightarrow \eta(x)$, so the conditional probability tends to $2\eta(x)(1 - \eta(x))$; integrate over the test point.

(c) Put $u = \min\{\eta, 1 - \eta\}$, so $L^* = \mathbf{E}[u(X)]$ and $2\eta(1 - \eta) = 2u(1 - u)$, the product being symmetric in η and $1 - \eta$. Since $u \leq 1/2$ we have $2(1 - u) \geq 1$, hence $2u(1 - u) \geq u$ and $R_1 \geq L^*$. The map $u \mapsto 2u(1 - u)$ is concave, so Jensen gives $R_1 \leq 2\mathbf{E}[u](1 - \mathbf{E}[u]) = 2L^*(1 - L^*) \leq 2L^*$.

```
1 R1 <- integrate(function(x) 2*eta(x)*(1-eta(x))/6, -3, 3)$value
2 c(L_star = L_star, R_1nn = R1, cover_hart = 2*L_star*(1-L_star),
3   twice_L = 2*L_star, observed_K1 = err[1, "test"])
```

```
##           L_star           R_1nn       cover_hart
##           0.1151           0.1658           0.2037
##      twice_L observed_K1.test
##           0.2302           0.1715
```

(d) The chain holds. The observed $K = 1$ error at $n_{\text{tr}} = 200$ sits above the limiting R_1 because the neighbour is not yet at x and the extra term in (a) is positive. Note $K = 1$ does **not** converge to L^* : that needs $K \rightarrow \infty$ with $K/n \rightarrow 0$. 

Exercise 1.10. At a test point a method has squared bias 0.09 and variance 0.16, with $\sigma^2 = 0.25$. Give the expected test MSE. A more flexible method halves the absolute bias and triples the variance: is it preferable? A third method has bias 0 and variance v : for which v does it beat

both?

Solution. By (9) the expected test MSE is $0.09 + 0.16 + 0.25 = 0.50$.

Halving the absolute bias quarters the squared bias to 0.0225; tripling the variance gives 0.48. The total is $0.7525 > 0.50$, so the flexible method is worse: the variance increase swamps the bias reduction. Only the two changed terms matter, σ^2 being common.

The unbiased method totals $v + 0.25$, beating the first when $v < 0.25$ and the second when $v < 0.5025$, hence both when $v < 0.25$. Unbiasedness is worth having only when its variance cost is below the squared bias plus variance it replaces. ■

Exercise 1.11. Theorem 1.13 assumes constant $\text{var}(\varepsilon)$. Suppose instead $\mathbf{E}[\varepsilon \mid X = x] = 0$ and $\text{var}(\varepsilon \mid X = x) = \sigma^2(x)$. State and prove what survives, at fixed x_0 and after averaging over an independent test point, and say what Corollary 1.14 becomes.

Solution. At fixed x_0 the proof is unchanged with σ^2 replaced by $\sigma^2(x_0)$: Step 1 needs only $\mathbf{E}[\varepsilon_0 \mid X_0 = x_0] = 0$ and independence of $\mathcal{D}$, so the cross term vanishes and the first square

contributes $\sigma^2(x_0)$; Step 2 involves $\mathcal{D}$ alone. Hence

$$\mathbf{E}[(Y_0 - \hat{f}(x_0))^2 \mid X_0 = x_0] = \text{var}(\hat{f}, x_0) + [\text{bias}(\hat{f}, x_0)]^2 + \sigma^2(x_0),$$

and averaging over X_0 , independent of $\mathcal{D}$,

$$\mathbf{E}[(Y_0 - \hat{f}(X_0))^2] = \mathbf{E}_{X_0}[\text{var}(\hat{f}, X_0)] + \mathbf{E}_{X_0}[\text{bias}(\hat{f}, X_0)^2] + \mathbf{E}[\sigma^2(X_0)].$$

Corollary 1.14 becomes $\mathbf{E}[(Y_0 - \hat{f}(X_0))^2] \geq \mathbf{E}[\sigma^2(X_0)]$. What is lost is a common pointwise floor: a method can be unimprovable in one region and far from it in another. Independence of ε and X , assumed in Definition 1.1, was never used; conditional mean zero suffices. ■

Exercise 1.12. Let $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ with $\mathbf{X}$ fixed of full column rank $k + 1$, $\mathbf{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$, $\text{var}(\boldsymbol{\varepsilon}) = \sigma^2\mathbf{I}$, and $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ the least-squares fit. (a) Show $\mathbf{E}[\text{MSE}_{\text{tr}}] = \sigma^2(n - k - 1)/n$. (b) For a fresh response $\mathbf{y}^{\text{new}} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}^{\text{new}}$ at the **same** $\mathbf{X}$, show $\mathbf{E}[n^{-1}\|\mathbf{y}^{\text{new}} - \hat{\mathbf{y}}\|^2] = \sigma^2(n + k + 1)/n$. (c) Deduce the optimism and evaluate the ratio of (b) to (a) for the fit of §5.6, which has $n = 506$, $k = 3$.

Solution. (a) $\mathbf{y} - \hat{\mathbf{y}} = (\mathbf{I} - \mathbf{H})\boldsymbol{\varepsilon}$ since $(\mathbf{I} - \mathbf{H})\mathbf{X} = \mathbf{0}$, so $\mathbf{E} \|(\mathbf{I} - \mathbf{H})\boldsymbol{\varepsilon}\|^2 = \sigma^2 \text{tr}(\mathbf{I} - \mathbf{H})$, the expectation of a quadratic form in a mean-zero vector of variance $\sigma^2 \mathbf{I}$. The prerequisites' theorem that a symmetric idempotent matrix has trace equal to rank gives $\text{tr}(\mathbf{I} - \mathbf{H}) = n - k - 1$.

(b) $\mathbf{y}^{\text{new}} - \hat{\mathbf{y}} = \boldsymbol{\varepsilon}^{\text{new}} - \mathbf{H}\boldsymbol{\varepsilon}$, two independent mean-zero terms, so $\mathbf{E} \|\mathbf{y}^{\text{new}} - \hat{\mathbf{y}}\|^2 = \sigma^2 n + \sigma^2 \text{tr}(\mathbf{H}) = \sigma^2(n + k + 1)$.

(c) The optimism is $2\sigma^2(k + 1)/n$: proportional to the number of fitted parameters and inversely to n . The ratio is $(n + k + 1)/(n - k - 1) = 510/502 = 1.0159$. The printed LOOCV against training ratio in §5.6 is 1.0338, about twice as large; the two are not the same quantity, since this calculation holds the design fixed and draws a fresh response, while LOOCV refits on $n - 1$ points and weights observation i by $(1 - h_{ii})^{-2}$. ■

Exercise 1.13. For the fit of §5.6, $\sum_i h_{ii} = k + 1$. (a) Prove it. (b) Show that if every leverage equalled $\bar{h} = (k + 1)/n$ then $\text{CV}_{(n)} = \text{MSE}_{\text{tr}}/(1 - \bar{h})^2$, and that this agrees with the optimism ratio of Exercise 1.12 to first order in $\bar{h}$. (c) Compare the equal-leverage prediction with the printed $\text{CV}_{(n)}$ and account for the gap.

Solution. (a) $\sum_i h_{ii} = \text{tr}\{\mathbf{X}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\} = \text{tr}\{(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{X}\} = \text{tr}(\mathbf{I}_{k+1}) = k + 1$ by the cyclic property of the trace.

(b) Substituting $h_{ii} = \bar{h}$ in (12) pulls the constant out. And $(1 - \bar{h})^{-2} = 1 + 2\bar{h} + O(\bar{h}^2)$ while $(n + k + 1)/(n - k - 1) = (1 + \bar{h})/(1 - \bar{h}) = 1 + 2\bar{h} + O(\bar{h}^2)$: two different arguments, one about refitting and one about a fresh response, agreeing to this order.

```
1 kp1 <- ncol(model.matrix(fit)); hbar <- kp1 / n
2 c(sum_h = sum(h), mean_h = mean(h), min_h = min(h), max_h = max(h),
3   equal_lev_CV = mean(e^2)/(1 - hbar)^2, actual_CV = cv_shortcut,
4   equal_ratio = 1/(1 - hbar)^2, actual_ratio = cv_shortcut/mean(e^2))
```

##	sum_h	mean_h	min_h	max_h
##	4.000000	0.007905	0.002052	0.077074
##	equal_lev_CV	actual_CV	equal_ratio	actual_ratio
##	30.960133	31.503994	1.016000	1.033847

(c) The equal-leverage calculation captures about half the inflation. The rest is the spread of the leverages: the largest is nearly ten times the mean, and $(1 - h)^{-2}$ is convex increasing, so replacing the leverages by their average understates the average of $(1 - h_{ii})^{-2}$. Cross-validated error is driven by the few high-leverage observations. 

Exercise 1.14. Fix $\lambda \geq 0$ and let $\hat{\boldsymbol{\beta}}(\lambda)$ minimise $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|^2$, with $\mathbf{X}$ of full column rank including its intercept column. Show that the leave-one-out shortcut survives verbatim with $\mathbf{A}_\lambda = \mathbf{X}'\mathbf{X} + \lambda\mathbf{I}$, $h_{ii}(\lambda) = \mathbf{x}_i'\mathbf{A}_\lambda^{-1}\mathbf{x}_i$ and $\hat{y}_i = \mathbf{x}_i'\hat{\boldsymbol{\beta}}(\lambda)$. What breaks if λ is re-selected for each deleted sample? Verify numerically at $\lambda = 50$.

Solution. The gradient of $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|^2$ vanishes at $\mathbf{A}_\lambda\hat{\boldsymbol{\beta}}(\lambda) = \mathbf{X}'\mathbf{y}$, the least-squares formula with $\mathbf{X}'\mathbf{X}$ replaced by $\mathbf{A}_\lambda$. Deleting observation i removes $\mathbf{x}_i\mathbf{x}_i'$ from $\mathbf{X}'\mathbf{X}$ and $\mathbf{x}_iy_i$ from $\mathbf{X}'\mathbf{y}$, so it replaces $\mathbf{A}_\lambda$ by $\mathbf{A}_\lambda - \mathbf{x}_i\mathbf{x}_i'$ and leaves λ alone. Every line of the proof of Theorem 1.18 goes through with $\mathbf{X}'\mathbf{X}$ rewritten as $\mathbf{A}_\lambda$: Lemma 1.17 with $\mathbf{A} = \mathbf{A}_\lambda$, $\mathbf{u} = \mathbf{v} = \mathbf{x}_i$ gives $\mathbf{v}'\mathbf{A}^{-1}\mathbf{u} = h_{ii}(\lambda)$, the deleted estimate is $\hat{\boldsymbol{\beta}}(\lambda) - \mathbf{A}_\lambda^{-1}\mathbf{x}_i(y_i - \hat{y}_i)/(1 - h_{ii}(\lambda))$, and premultiplying by $\mathbf{x}_i'$ gives

$$y_i - \mathbf{x}_i'\hat{\boldsymbol{\beta}}^{(-i)}(\lambda) = \frac{y_i - \hat{y}_i}{1 - h_{ii}(\lambda)}.$$

Nothing used the projection property of $\mathbf{H}$, only the two rank-one updates.

```
1 Xr <- model.matrix(fit); yr <- Boston$medv; lam <- 50
2 A <- crossprod(Xr) + lam * diag(ncol(Xr))
3 bh <- solve(A, crossprod(Xr, yr)); Ai <- solve(A)
```

```

4 h1 <- rowSums((Xr %*% Ai) * Xr)                                #  $h_{ii}(\lambda)$ 
5 yh <- as.vector(Xr %*% bh)
6 brute <- sapply(seq_len(n), function(i) {
7   Ai2 <- crossprod(Xr[-i, ]) + lam * diag(ncol(Xr))
8   as.vector(Xr[i, ] %*% solve(Ai2, crossprod(Xr[-i, ], yr[-i])))
9 })
10 c(max_abs_diff = max(abs((yr - brute) - (yr - yh)/(1 - h1))),
11   CV_ridge_shortcut = mean(((yr - yh)/(1 - h1))^2),
12   CV_ridge_brute     = mean((yr - brute)^2), CV_ols = cv_shortcut)

```

```

##      max_abs_diff CV_ridge_shortcut      CV_ridge_brute
##      8.349e-14      3.118e+01      3.118e+01
##      CV_ols
##      3.150e+01

```

If λ is re-chosen on each deleted sample then $\mathbf{A}_\lambda$ itself changes with i and the update is no longer rank one. This is §6.2 again: the object being cross-validated must be the whole procedure, and once tuning is part of it there is no shortcut. ■

Exercise 1.15. State how many models the validation-set approach, LOOCV and k -fold cross-validation fit on n observations, and what Theorem 1.18 changes for least squares. Then count the fits for the honest procedure of §6.2 when a tuning parameter with m candidate values is selected by V -fold cross-validation inside each of K outer folds, and evaluate at $K = V = 10$, $m = 20$.

Solution. One, n and k . For least squares, Theorem 1.18 reduces LOOCV to **one** fit, $CV_{(n)}$ being a function of the full-sample residuals and leverages alone. The saving does not extend to logistic regression or a tree.

Each outer fold runs an inner V -fold selection over m candidates at Vm fits, then refits once on the outer training set: $K(Vm + 1)$, plus a final fit on all n , so $K(Vm + 1) + 1$. At $K = V = 10$, $m = 20$ that is $10(200 + 1) + 1 = 2011$ fits for one honest error estimate. ■

Exercise 1.16. LOOCV uses $n - 1$ observations per fit and 10-fold uses $0.9n$, so LOOCV is the less biased. Why is 10-fold nonetheless often the more accurate estimate of test error?

Solution. Bias is not the only component of the error of the **estimate**. The n quantities LOOCV averages come from training sets overlapping in $n - 2$ observations, so they are highly positively

correlated, and the variance of a mean of highly correlated terms does not fall like $1/n$. The 10 quantities in 10-fold come from fits overlapping much less, so their average has lower variance. The slightly larger upward bias of 10-fold is usually the better trade.

The variance here is over the draw of the **training sample**, not over the choice of split with the sample held fixed. The two behave differently in k and must not be confused: the second vanishes at $k = n$, where only one partition exists, while the first does not. ■

Exercise 1.17. Repeat the experiment of §5.7 at $k = 2, 5, 10, 25$, thirty times each on the fixed **Boston** sample, and report the mean and standard deviation with the LOOCV value. Explain the pattern in the standard deviations and why it does not contradict Exercise 1.16.

Solution.

```
1 set.seed(11)
2 tab <- t(sapply(c(2, 5, 10, 25), function(K) {
3   r <- replicate(30, cv_kfold(K)); c(k = K, mean = mean(r), sd = sd(r))
4 })))
5 rbind(round(tab, 4), c(k = n, mean = round(cv_shortcut, 4), sd = 0))
```

```
##          k  mean      sd
## [1,]     2 32.41  1.5394
## [2,]     5 31.62  0.4241
## [3,]    10 31.47  0.3558
## [4,]    25 31.59  0.2037
## [5,]   506 31.50  0.0000
```

The standard deviation falls monotonically to zero and the mean at $k = 2$ is the largest: each 2-fold fit uses only 253 observations, so the upward bias from training on less data is worst there, as §4.3 predicts.

These standard deviations measure variability over the random **split** with the sample fixed. As k grows the number of admissible partitions falls, and at $k = n$ there is only one, so this variability must vanish. It says nothing about how the estimate would move under a new sample of 506 houses, which is the variance Exercise 1.16 concerns and which is not zero for LOOCV. A cross-validated figure quoted with a standard deviation of 0 is not a precise estimate of test error. ■

Exercise 1.18. Repeat the bias–variance computation of §5.5 at $x_0 = 2.9$ instead of $x_0 = 1$. Report the four columns, check the identity, and explain the difference.

Solution.

```

1  set.seed(5)
2  bv29 <- t(sapply(degs[1:6], function(dg) {
3    fh <- y0 <- numeric(B)
4    for (b in seq_len(B)) {
5      d <- data.frame(x = runif(nsim, -3, 3))
6      d$y <- f(d$x) + rnorm(nsim, sd = sigma)
7      fh[b] <- predict(lm(y ~ poly(x, dg), data = d),
8                      newdata = data.frame(x = 2.9))
9      y0[b] <- f(2.9) + rnorm(1, sd = sigma)
10   }
11   b2 <- (mean(fh) - f(2.9))^2; v <- var(fh)
12   c(bias2 = b2, variance = v, sum = b2 + v + sigma^2,
13     direct = mean((y0 - fh)^2))
14 })))
15 cbind(degree = degs[1:6], round(bv29, 4))

```

##	degree	bias2	variance	sum	direct
## [1,]	1	0.0005	0.0342	0.2847	0.2954
## [2,]	2	0.0000	0.0697	0.3197	0.3132

```
## [3,]      4 0.9136    0.5181    1.6816    1.6253
## [4,]      6 0.1237    0.4841    0.8578    0.8709
## [5,]      8 0.0040    9.5742    9.8282    9.8893
## [6,]     12 0.0135   31.8462   32.1097   31.8067
```

sum and **direct** still agree to Monte-Carlo error, so Theorem 1.13 is an identity at every x_0 and not a statement about typical points. What changes is scale: the degree-12 variance is 0.044 at the interior point $x_0 = 1$ and is 730 times larger at $x_0 = 2.9$, a distance 0.1 from the edge of the support. A least-squares polynomial is determined by the whole sample, and near the boundary there are training points on one side only, so the fit extrapolates and coefficient errors are amplified. Consequences: the U-shape in flexibility is much sharper at the boundary, with the best degree 1 or 2 rather than 8; and a method compared at interior points only will be flattered. ■

Exercise 1.19 (ISLR §5.4, exercise 8). Generate $n = 100$ observations from $y = x - 2x^2 + \varepsilon$ with x, ε standard normal, and compute the LOOCV error for polynomials of degree 1 to 4. (a) Which degree minimises it, and is that expected? (b) Repeat with a different seed and explain the result. (c) Compare with the t statistics of the degree-4 fit.

Solution.

```

1  set.seed(1)
2  xs <- rnorm(100); ys <- xs - 2*xs^2 + rnorm(100)
3  ds <- data.frame(x = xs, y = ys)
4  cvp <- sapply(1:4, function(dg)
5    cv.glm(ds, glm(y ~ poly(x, dg), data = ds))$delta[1])
6  set.seed(99)
7  cvp2 <- sapply(1:4, function(dg)
8    cv.glm(ds, glm(y ~ poly(x, dg), data = ds))$delta[1])
9  rbind(degree = 1:4, seed_1 = round(cvp, 4), seed_99 = round(cvp2, 4))

```

```

##           [,1]    [,2]    [,3]    [,4]
## degree  1.000 2.0000 3.0000 4.0000
## seed_1   7.288 0.9374 0.9566 0.9539
## seed_99  7.288 0.9374 0.9566 0.9539

```

(a) Degree 2, and that is expected: the truth is quadratic, so degree 1 carries a large squared bias and degrees 3 and 4 add variance without removing bias. The drop from degree 1 to 2 is by a factor of about eight; from 2 to 4 the estimate moves by less than 2%.

(b) The two rows are identical. LOOCV involves no random split, the n training sets being

determined by the data, so the seed cannot matter. Only k -fold with $k < n$ has split variability (Exercise 1.17).

```
1 round(coef(summary(lm(y ~ poly(x, 4), data = ds))), 4)
```

```
##              Estimate Std. Error  t value Pr(>|t|)
## (Intercept)  -1.5500      0.0959 -16.1620  0.0000
## poly(x, 4)1   6.1888      0.9591   6.4531  0.0000
## poly(x, 4)2 -23.9483      0.9591 -24.9708  0.0000
## poly(x, 4)3   0.2641      0.9591   0.2754  0.7836
## poly(x, 4)4   1.2571      0.9591   1.3108  0.1931
```

(c) The linear and quadratic terms are overwhelmingly significant and the cubic and quartic are not, which is the same conclusion the cross-validation reached. The two agree here because the candidate models are nested and the design is orthogonal in **poly**; they need not agree in general, since a t statistic tests a coefficient and cross-validation estimates prediction error. ■

Exercise 1.20. A colleague screens 10,000 candidate predictors on the full data, keeps the 20 most correlated with the response, then reports a 10-fold cross-validated error for a model using those 20. What is wrong, and what would you do instead? With $n = 100$ and no signal, how large

a correlation should the best of the 10,000 show?

Solution. The screening used every observation, including those in each held-out fold, so the folds are not independent of the fitting procedure and the reported error is optimistic (§6.2). The whole procedure must be repeated inside each fold: for each j , screen using only the other $k - 1$ folds, fit on those, and predict fold j . The quantity cross-validated is the **procedure**, not the final model; Exercise 1.15 counts the cost.

Each sample correlation of an independent pair is approximately $N(0, 1/(n-1))$, and the maximum of p roughly independent standard normals is close to $\sqrt{2 \ln p}$, so $\max_j |r_j| \approx \sqrt{2 \ln p} / \sqrt{n-1}$.

```
1 set.seed(13)
2 mx <- replicate(20, max(abs(cor(matrix(rnorm(100*10000), 100, 10000),
3                                   rnorm(100)))))
4 c(mean_max_abs_r = mean(mx), sd = sd(mx),
5   extreme_value_approx = sqrt(2*log(10000))/sqrt(99))
```

```
##           mean_max_abs_r           sd
##           0.38452         0.03085
## extreme_value_approx
```

```
## 0.43136
```

The approximation overstates it slightly, the extreme-value limit being approached slowly. Either way the colleague finds predictors correlating around 0.4 with the response on pure noise, which is why the screened model looks convincing and why §6.2 reports a cross-validated error below $\text{var}(y)$. ■

Exercise 1.21. Simulate 100 groups of 10 observations with $y_{gi} = a_g + \varepsilon_{gi}$, the group effects a_g independent $N(0, 4)$ and ε_{gi} independent $N(0, 1)$, and let the single predictor be the group label, $x_{gi} = g/100$ jittered by $N(0, 0.001^2)$. Estimate the error of a 1-nearest-neighbour rule by 10-fold cross-validation splitting on rows and again splitting on groups. Predict both numbers first.

Solution. Under a row split, nine members of a held-out observation's group are in the training set at essentially the same x , so the neighbour is a sibling: the prediction is $a_g + \varepsilon_{gj}$ against a truth of $a_g + \varepsilon_{gi}$ and the expected squared error is $2\sigma_\varepsilon^2 = 2$, with the group effect never tested. Under a group split the neighbour comes from an adjacent group with an independent $a_{g'}$, giving $2\text{var}(a) + 2\sigma_\varepsilon^2 \approx 10$.

```
1 set.seed(41)
```

```

2 G <- 100; mg <- 10; ng <- G*mg; grp <- rep(1:G, each = mg)
3 a <- rnorm(G, sd = 2)
4 xg <- grp/G + rnorm(ng, sd = 0.001)
5 yg <- a[grp] + rnorm(ng, sd = 1)
6 nn1 <- function(xtr, ytr, xte)
7   ytr[max.col(-abs(outer(xte, xtr, "-")), "first")]
8 cvg <- function(byGroup, K = 10) {
9   fo <- if (byGroup) sample(rep(seq_len(K), length.out = G))[grp]
10     else             sample(rep(seq_len(K), length.out = ng))
11   mean(vapply(seq_len(K), function(j)
12     mean((yg[fo == j] - nn1(xg[fo != j], yg[fo != j], xg[fo == j]))^2),
13     numeric(1)))
14 }
15 c(row_split = mean(replicate(20, cvg(FALSE))),
16   group_split = mean(replicate(20, cvg(TRUE))),
17   var_a = var(a), var_y = var(yg))

```

```
## row_split group_split var_a var_y
##      2.158      10.565    4.229    5.054
```

Both predictions hold. The row split reports an error well **below** $\text{var}(y)$ and would be read as a

good model; the group split reports twice that variance, so on a new group the rule is worse than predicting the overall mean. The predictor carries no information about a_g : it is an identifier, and a random row split lets the rule look it up. Ask what unit is independent (§6.3); here it is the group, so the fold must be the group. ■

Exercise 1.22. Let $q_n = (1 - 1/n)^n$ be the out-of-bag probability of Proposition 1.22. (a) Prove q_n increases in n , using the series for $\ln(1 - u)$. (b) Show $e^{-1} - q_n \approx e^{-1}/(2n)$ and tabulate both at $n = 5, 10, 100, 506$. (c) §5.8 reports an empirical fraction of 0.3677 at $n = 506$ from 2000 bootstrap samples. Is that consistent with q_{506} ?

Solution. (a) For $n \geq 2$,

$$\ln q_n = n \ln \left(1 - \frac{1}{n}\right) = -n \sum_{j \geq 1} \frac{1}{j n^j} = -1 - \frac{1}{2n} - \frac{1}{3n^2} - \cdots,$$

every term after the first being positive and decreasing in n . So $\ln q_n$ increases and tends to -1 , which is Proposition 1.22.

(b) From the display, $q_n = e^{-1} \{1 - 1/(2n) + O(n^{-2})\}$.

```

1 nn <- c(5, 10, 100, 506); qn <- (1 - 1/nn)^nn
2 rbind(n = nn, q_n = qn, gap = exp(-1) - qn, approx = exp(-1)/(2*nn))

```

```

##           [,1]      [,2]      [,3]      [,4]
## n         5.00000 10.00000 100.000000 506.000000
## q_n        0.32768 0.34868 0.366032 0.3675156
## gap        0.04020 0.01920 0.001847 0.0003638
## approx     0.03679 0.01839 0.001839 0.0003635

```

The approximation is within 9% at $n = 5$ and 0.1% by $n = 506$: the convergence is of order $1/n$, so “roughly a third” is already accurate at $n = 10$.

(c) Each replicate averages 506 indicators. Its mean has standard deviation at most 0.0214, since $\sqrt{q(1-q)/506} = 0.0214$; averaging 2000 replicates reduces this bound to 0.00048. Within one replicate the indicators are negatively correlated because $(1 - 2/n)^n < (1 - 1/n)^{2n}$, which only reduces the variance. The gap $0.3677 - 0.36752 = 0.00018$ is well inside that bound. ■

Exercise 1.23. Let $\hat{\theta}$ be the sample mean. (a) Show the **ideal** bootstrap standard error, the exact standard deviation of $\hat{\theta}^*$ under resampling, is $\hat{\sigma}/\sqrt{n}$ with $\hat{\sigma}^2 = n^{-1} \sum_i (x_i - \bar{x})^2$, and compare

with $s/\sqrt{n}$. (b) Verify by exhaustive enumeration on $\{2, 5, 8\}$. (c) How many distinct resamples are there, and why can they not be enumerated with equal weights?

Solution. (a) A bootstrap sample draws n values independently from the empirical distribution, which has mean $\bar{x}$ and variance $\hat{\sigma}^2$ (divisor n : it is the variance of a law putting mass $1/n$ at each x_i). The mean of n independent draws therefore has variance $\hat{\sigma}^2/n$ exactly, so

$$\text{SE}_{\infty}(\hat{\theta}) = \frac{\hat{\sigma}}{\sqrt{n}} = \sqrt{\frac{n-1}{n}} \cdot \frac{s}{\sqrt{n}},$$

smaller than the textbook standard error by $\sqrt{(n-1)/n}$, and requiring no simulation. SE_B of (14) estimates this with a Monte-Carlo error that vanishes as $B \rightarrow \infty$ and has nothing to do with the sampling error of $\hat{\theta}$.

```

1 dv <- c(2, 5, 8); nb <- 3
2 mns <- rowMeans(as.matrix(expand.grid(rep(list(dv), nb))))
3 c(n_ordered = length(mns), n_multisets = choose(2*nb - 1, nb),
4   E_star = mean(mns), sd_star = sqrt(mean((mns - mean(mns))^2)),
5   sigmahat_over_sqrt_n = sqrt(mean((dv - mean(dv))^2))/sqrt(nb),
```

```
6 s_over_sqrt_n = sd(dv)/sqrt(nb), ratio = sqrt((nb - 1)/nb))
```

```
##           n_ordered           n_multisets
##           27.0000           10.0000
##           E_star           sd_star
##           5.0000           1.4142
## sigmahat_over_sqrt_n      s_over_sqrt_n
##           1.4142           1.7321
##           ratio
##           0.8165
```

(b) With $n = 3$ there are $3^3 = 27$ equally likely ordered resamples, whose means average to $\bar{x} = 5$ with standard deviation $\sqrt{2}$; and $\hat{\sigma}^2 = 6$ gives $\hat{\sigma}/\sqrt{3} = \sqrt{2}$ exactly, against $s/\sqrt{3} = \sqrt{3}$.

(c) The distinct **multisets** number $\binom{2n-1}{n} = 10$, but they are not equally likely: $\{2, 5, 8\}$ arises in $3! = 6$ of the 27 orderings and $\{2, 2, 2\}$ in one. Enumerating the ten with equal weights gives the wrong answer. The n^n ordered draws are the equally likely objects, which is why enumeration is abandoned for B random draws at any realistic n . ■

Exercise 1.24 (ISLR §5.4, exercise 1). Two assets have returns X and Y . Investing a fraction α in X and $1 - \alpha$ in Y gives variance $\text{var}(\alpha X + (1 - \alpha)Y)$. (a) Derive the minimising α . (b) Estimate it on `ISLR2::Portfolio` and give a bootstrap standard error. (c) The plug-in $\hat{\alpha}$ is a ratio of estimated variances and covariances, so no standard-error formula is elementary. Say why the bootstrap is the right tool here.

Solution. (a) Writing $\sigma_X^2, \sigma_Y^2, \sigma_{XY}$ for the two variances and the covariance,

$$v(\alpha) = \alpha^2 \sigma_X^2 + (1 - \alpha)^2 \sigma_Y^2 + 2\alpha(1 - \alpha)\sigma_{XY}.$$

Then $v'(\alpha) = 2\alpha\sigma_X^2 - 2(1 - \alpha)\sigma_Y^2 + 2(1 - 2\alpha)\sigma_{XY}$, and setting it to zero,

$$\alpha^* = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}.$$

The denominator is $\text{var}(X - Y) > 0$ unless $X - Y$ is degenerate, and $v''(\alpha) = 2\text{var}(X - Y) > 0$, so this is the unique minimum.

```
1 P <- ISLR2::Portfolio
2 alpha_hat <- function(d, i) {
```

```

3   vx <- var(d$X[i]); vy <- var(d$Y[i]); cxy <- cov(d$X[i], d$Y[i])
4   (vy - cxy) / (vx + vy - 2*cxy)
5 }
6 set.seed(7)
7 ba <- boot(P, alpha_hat, R = 1000)
8 c(alpha_hat = alpha_hat(P, seq_len(nrow(P))), boot_se = sd(ba$t),
9   var_at_alpha = with(P, {
10     a <- alpha_hat(P, seq_len(nrow(P)))
11     a^2*var(X) + (1-a)^2*var(Y) + 2*a*(1-a)*cov(X, Y) }),
12   var_at_half = with(P, 0.25*var(X) + 0.25*var(Y) + 0.5*cov(X, Y)))

```

```

##      alpha_hat      boot_se var_at_alpha  var_at_half
##      0.57583      0.08978      0.91559      0.92240

```

(c) $\hat{\alpha}$ is a smooth but non-linear function of three second moments, so its sampling distribution has no closed form at finite n ; a delta-method expansion would need fourth moments and would still be an approximation. Resampling the 100 observed pairs assumes only that they are an independent sample, which is the point made in the remark following Definition 1.21. The gain over the naive $\alpha = 1/2$ is small here, which the standard error puts in context. ■

Exercise 1.25 (ISLR §5.4, exercise 9). For `medv` in `Boston`: (a) estimate the population mean and give its standard error from the usual formula and from the bootstrap; (b) form a bootstrap confidence interval and compare with `t.test`; (c) estimate the standard errors of the median and of the tenth percentile, and say why no formula was used.

Solution.

```

1 mv <- Boston$medv; nB2 <- length(mv)
2 set.seed(3)
3 b_mean <- boot(mv, function(v, i) mean(v[i]),          R = 2000)
4 b_med  <- boot(mv, function(v, i) median(v[i]),        R = 2000)
5 b_q10  <- boot(mv, function(v, i) quantile(v[i], 0.1), R = 2000)
6 rbind(estimate = c(mean = mean(mv), median = median(mv),
7                     q10 = unname(quantile(mv, 0.1))),
8       boot_se   = c(sd(b_mean$t), sd(b_med$t), sd(b_q10$t)),
9       formula   = c(sd(mv)/sqrt(nB2), NA, NA))

```

```

##           mean  median   q10
## estimate 22.5328 21.2000 12.750
## boot_se  0.4139  0.3773  0.504
## formula  0.4089      NA     NA

```

```
1 rbind(boot_CI = mean(mv) + c(-2, 2)*sd(b_mean$t),  
2       t_test  = as.numeric(t.test(mv)$conf.int))
```

```
##           [,1]  [,2]  
## boot_CI 21.71 23.36  
## t_test  21.73 23.34
```

(a) The bootstrap standard error of the mean is within 1.5% of $s/\sqrt{n}$. That is the expected agreement: Exercise 1.23 shows the two differ in the population only by the factor $\sqrt{(n-1)/n}$, which is 0.999 here, the rest being Monte-Carlo error at $B = 2000$.

(b) The two intervals agree to about 0.03 on each end. The bootstrap interval uses the normal approximation $\hat{\mu} \pm 2\text{SE}$ while `t.test` uses t_{505} and $s/\sqrt{n}$; at $n = 506$ the two multipliers are 2 and 1.965.

(c) The sample median and the tenth percentile are not smooth functions of the data, and their standard errors involve the unknown density of the population at those quantiles. The bootstrap needs neither. Note the tenth percentile has the largest standard error of the three, the density of `medv` being thinner in the lower tail than at the centre. ■

Exercise 1.26. For $(x, y) = (\text{lstat}, \text{medv})$ in **Boston**, bootstrap the sample correlation with $B = 2000$ and compare with the normal-theory standard error $(1 - r^2)/\sqrt{n}$, which the delta method of the prerequisites gives for a bivariate normal population. Which do you report?

Solution.

```

1 set.seed(14)
2 bc <- boot(Boston[, c("lstat", "medv")],
3           function(d, i) cor(d[i, 1], d[i, 2]), R = 2000)
4 r_hat <- cor(Boston$lstat, Boston$medv)
5 c(r = r_hat, boot_se = sd(bc$t), delta_se = (1 - r_hat^2)/sqrt(nB2),
6   ratio = ((1 - r_hat^2)/sqrt(nB2))/sd(bc$t),
7   medv_at_50 = sum(Boston$medv == 50), skew_medv = skew(Boston$medv))

```

```

##           r      boot_se   delta_se      ratio medv_at_50
##  -0.73766    0.01569    0.02027    1.29185    16.00000
## skew_medv
##    1.10154

```

The formula is not wrong; its hypothesis is. It is the delta method applied to a bivariate **normal** population, where $\sqrt{n}(r - \rho) \xrightarrow{d} N(0, (1 - \rho^2)^2)$. This population is not bivariate normal: **medv**

is censored, with 16 of 506 values sitting exactly at 50, and its skewness is 1.10. The bootstrap assumes only that the 506 pairs are an independent sample, so it is the figure to report, and the discrepancy measures the departure from normality. In §5.8 the disagreement went the other way, the formula being the optimistic one because it assumed constant variance. ■